Adaptive Decision-Making in Attack-Defense Games With Bayesian Inference of Rationality Level

Hongwei Fang and Peng Yi

***Abstract*—This article investigates two-player attack-defense (AD) games involving players with bounded rationality, where the defender aims to intercept the attacker, while the attacker aims to invade the protected area and avoid interception. We first set path planning optimization problems in a receding horizon fashion for each player and formulate the AD game. Then, using the level-k model of behavioral game theory, we specify the decision mechanisms for players with bounded rationality. We propose an adaptive path planning strategy, coupled with the Bayesian learning method, for the defender to counter the attacker with an unknown reasoning level of the decision mechanism. The Bayesian inference algorithm, which combines current observation information and historical receding horizon prediction trajectories to form the belief on the attacker's reasoning level, allows the defender to generate an adaptive interception trajectory with the multimodel strategy. Finally, both numerical simulations and experiments confirm the effectiveness of the proposed algorithm.**

***Index Terms*—Attack-defense (AD) game, Bayesian learning, level-k theory, receding horizon optimization.**

I. Introduction

ADVERSARIAL games are an important type of game model used to describe the interactive behavior between opposing players. In such a pursuit, each player strives to maximize his utility, and compete against the other. Typical adversarial game models include the following categories: attack-defense (AD) games [1], [2], pursuit-evasion games [3], [4], herding games [5], [6], target-attacker-defender games [7], and racing games [8], [9]. So far, these game models have served as standard typical frameworks for designing effective decision-making mechanisms in various practical contexts, including the pursuit of criminals, bird tracking, network security, and national security.

Constructing the mechanism of an opponent's behavior and predicting his future strategies is a significant challenge in adversarial games. The widely used prediction model is based on the concept of Nash equilibrium (NE) [10], [11], [12], called equilibrium model. It assumes that all players have the same decision-making mechanism and are aware of the full information of the players participating in the game. In this case, the NE strategy should be adopted by all players, and no player has an incentive to change its strategy unilaterally. However, in many games where the decision-making process of the opponents is opaque, or where human players are involved, equilibrium models may not be able to predict or explain the observed behavior. One reason is that players are boundedly rational and do not have sufficient depth of reasoning to reach the NE. Meanwhile, players may not be at the same level of reasoning and have the same information about the game. Recent experimental studies [13], [14] suggest that in the presence of asymmetric information and asymmetric reasoning level, a nonequilibrium model provides a more accurate description of player behavior than an equilibrium model. A nonequilibrium model allows players to adopt strategies that are not the best response to other players' strategies, resulting in deviations from equilibrium. When studying players with different levels of computation and reasoning ability, it is essential to investigate solutions that consider bounded rationality.

A. Related Work

Currently, most research on adversarial games employs NE as the expected solution. For example, in [6], defence robots apply NE strategies to repel intruders toward a safe zone and prevent them from entering one or more protected areas. In [8] and [9], players use an iterative best response (IBR) algorithm to calculate NE in real time. This technique allows players to predict their opponents' actions with the NE and plan their own future trajectories accordingly. Other studies use differential game approaches [3], [7], where the adversarial interactions between players are considered as a dynamic game subject to

Manuscript received 5 October 2023; revised 23 February 2024; accepted 5 April 2024. Date of publication 29 May 2024; date of current version 11 October 2024. *(Corresponding author: Peng Yi.)*

Hongwei Fang is with the Department of Control Science and Engineering, Tongji University, Shanghai 201804, China (e-mail: hongweif@tongji.edu.cn).

Peng Yi is with Department of Control Science and Engineering, Tongji University, Shanghai 201804, China, and also with the National Key Laboratory of Autonomous Intelligent Unmanned Systems, Shanghai Research Institute for Intelligent Autonomous Systems, and Frontiers Science Center for Intelligent Autonomous Systems, Ministry of Education, Tongji University, Shanghai 201804, China (e-mail: yipeng@tongji.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TIE.2024.3393118.

Digital Object Identifier 10.1109/TIE.2024.3393118

0278-0046 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

their kinodynamic constraints. Such methods also follow the equilibrium strategy to find a solution.

Unlike equilibrium models, nonequilibrium models do not assume complete rationality, nor do they go beyond simple stimulus-response decision mechanisms. Instead, they allow players to make conscious but imperfect attempts to achieve favorable returns [15], [16], [17]. These models also examine how the behavior of one player can affect the decisions of others, and specify the players' prediction and decision mechanisms. Therefore, nonequilibrium models are applicable to games involving players who do not adopt NE strategies. For example, [18] discovers that if one player adopts a fixed nonequilibrium strategy (such as pure pursuit or pure evasion), the other player can improve his performance by adopting nonequilibrium strategies as well. Currently, most research on nonequilibrium theory has concentrated on players with bounded rationality, for example, [19] studies the construction of a framework with limited rationality and proposes reinforcement learning (RL) approaches for experimental games. Furthermore, structural nonequilibrium models have been examined in [20] and applied to behavioral training for autonomous vehicles in [21].

Another category of nonequilibrium models relies on a finite iteration of best response strategies up to a certain level. The level-k game theory assumes that each player believes that all his opponents are level-(k-1) players [22]. Therefore, level-k players no longer seek (Nash) equilibrium but strive to make the best response to their level-(k-1) opponents. This level-k theory is frequently employed by researchers to simulate interactions between autonomous agents with bounded rationality, such as humans. In some studies [23], [24], [25], [26], the level-k method is used to simulate vehicle behavior during events such as lane changes, lane merges, and intersections. This method takes into account the future actions of neighboring vehicles under different levels of reasoning. Furthermore, the study [4] applies the level-k model to represent players with bounded rationality in pursuit-evasion games.

However, in real-world adversarial games, players often cannot know the exact level of their opponents, thus they must infer their opponents' levels before making a response. The authors in [23], [24], [25], and [26] use the one-step differences between the predicted trajectories of different level-k decision models and the observed trajectory to update the belief of each model at each instant. The probability distribution of the models is used to create a multimodel decision strategy, which assigns a player to multiple levels in the hierarchical structure with an inference belief [24]. The aforementioned level inference approach is valid for agents that are constrained by traffic rules (lane changes for overtaking or turning, etc.), because their decision-making level can be easily characterized by observed driving behaviors, such as driving along a lane, changing lanes or executing a parking maneuver [27]. But in AD games, where players are free to move around, the decision outcomes at different levels of reasoning may be similar or identical. Hence, the one-step difference information is not sufficient to characterize the feature of a level-k player. Therefore, this article proposes an online Bayesian inference algorithm that makes extensive use of past historical prediction trajectories and current observations. It updates the probability distribution of k level reasoning models by evaluating the posterior probability of the sampled historical k level prediction trajectories with the opponent's actual motion trajectory. This approach captures the behavioral characteristics of the k level player, and gains better inference accuracy through continuous interactions. It improves the defender's win rate combined with a multimodel strategy.

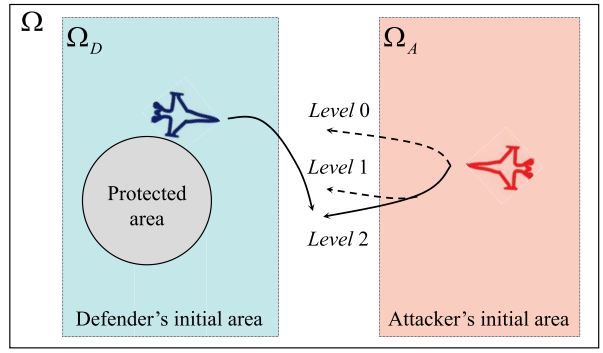


Fig. 1. AD game. The attacker is a decision maker with bounded rationality, and different rational decision levels produce different intrusion trajectories. The defender must infer the decision level of the attacker and then generate interception trajectories.

B. Our Work

This article focuses on the two-player AD game with bounded rationality. We present an adaptive path planning algorithm based on a Bayesian inference technique for the defender. The main contributions of this article are as follows: 1) In contrast to the most AD game models that concentrate on equilibrium strategies, we construct a nonequilibrium AD game model with level-k rationality in a receding horizon manner. This model is applicable to adversarial games involving players with different computation and reasoning abilities; 2) Based on the defender's historical prediction of the attacker's level-k trajectory, we design a Bayesian inference algorithm for the defender to update the beliefs of the attacker's reasoning levels. By combining a multimodel strategy, this strategy improves the defender's win rate; and 3) We not only verify the effectiveness of the algorithm in numerical simulations, but also design an experiment framework based on our hardware platform to test it. The proposed method is able to infer the opponent's reasoning level online and generate countering strategies.

The rest of this article is organized as follows. In Section II, we present the problem statement. In Section III, the level-k game-theoretic decision mechanism is presented. In Section IV, the Bayesian inference technique is constructed and the adaptive path planning strategy is developed. In Section V, the simulations and experiments are performed to validate the proposed method. Finally, in Section VI, the conclusion and future work are summarized.

II. Problem Formulation

Consider a two-player AD game scenario as shown in Fig. 1. The game space is represented as $\Omega \in \mathbb{R}^n$, consisting of a protected area, an attacker, and a defender, where n denotes the

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MAIN SYMBOLS

Symbols	Meaning
$V=\{D,A\}$	Set of two players, D denotes the defender and A denotes the attacker.
$\mathcal{X}_j \subset \mathbb{R}^{2n}$	Player $j's$ state set, $\forall j \in V$.
$\mathcal{Y}_j \subset \mathbb{R}^{n}$	Player $j's$ position set, $\forall j \in V$.
$\mathcal{U}_j \subset \mathbb{R}^{n}$	Player $j's$ action set, $\forall j \in V$.
$J_j : \mathbb{R}^{2n} \to \mathbb{R}$	Player $j's$ cost function, $\forall j \in V$.
$\mathcal{C}_j : \mathbb{R}^{2n} \to \mathbb{R}^{n}$	Player $j's$ dynamics constraint, $\forall j \in V$.
$H \in \mathbb{R}$	Receding horizon prediction step.
$\mathbf{u}_j^t \in \mathcal{U}_j$	Player $j's$ action at time t, representing accelerations along the n-dimensional axis, $\forall j \in V$. For example, if $n=3$, $\mathbf{u}_j^t = [a_{x,j}^t, a_{y,j}^t, a_{z,j}^t]^T$ denotes the acceleration vector that contains the components in the x, y and z directions.
$\mathbf{u}_{-j}^t \in \mathcal{U}_{-j}$	The other player action except player j.
$\mathbf{x}_j^t \in \mathcal{X}_j$	Player $j's$ state, which contains positions and velocities along the n-dimensional axis, $\forall j \in V$. For example, if $n=3$, $\mathbf{x}_j^t = [x_j^t, y_j^t, z_j^t, v_{x,j}^t, v_{y,j}^t, v_{z,j}^t]^T$ denotes the state vector that contains position and velocity components in the x, y and z directions.
$\mathbf{x}_{-j}^t \in \mathcal{X}_{-j} := \prod_{j \in V \setminus \{j\}} \mathcal{X}_j$	The other player state except player j.
$\mathbf{y}_j^t \in \mathcal{Y}_j$	Player $j's$ position.
$\mathbf{y}_{-j}^t \in \mathcal{Y}_{-j} := \prod_{j \in V \setminus \{j\}} \mathcal{Y}_j$	The other player position except player j.

dimension of the game space ($n=2$ and 3 are considered in the article). The initial positions of the attacker and the defender are within the regions $\Omega_D \in \mathbb{R}^n$ and $\Omega_A \in \mathbb{R}^n$, respectively. The defender's goal is to intercept the attacker, while the attacker's goal is to invade the protected area and avoid the defender. In the game, if the attacker approaches the protected area, the attacker wins. Conversely, if the defender intercepts the attacker, the defender wins. The following parameters are defined as shown in Table I.

A. Dynamics

For player j, given the current state $\mathbf{x}_j^t$ and strategy $\mathbf{u}_j^t$ at time t, we can compute his subsequent state using a second-order dynamics system. The discrete-time dynamics for player j is expressed as follows:

$$\begin{aligned} \mathbf{x}_j^{t+1} &= \mathbf{A}\mathbf{x}_j^t + \mathbf{B}\mathbf{u}_j^t \\ \mathbf{y}_j^t &= \mathbf{C}\mathbf{x}_j^t, \quad \forall j \in V. \end{aligned} \tag{1}$$

where $\mathbf{A}=[\mathbf{I}_n, T\times\mathbf{I}_n; 0\times\mathbf{I}_n, \mathbf{I}_n]\in\mathbb{R}^{2n\times 2n}$, $\mathbf{B}=[T^2/2\times\mathbf{I}_n; T\times\mathbf{I}_n]\in\mathbb{R}^{2n\times n}$, and $\mathbf{C}=[\mathbf{I}_n, 0\times\mathbf{I}_n]^T\in\mathbb{R}^{2n\times 2n}$, $\mathbf{I}_n$ is a n-dimensional unit diagonal matrix.

B. Cost Function

For each player, considering the roles they play in the game, we construct the following cost functions.

Defender: The purpose is to protect the protected area and intercept the attacker, so the cost function is defined as

$$J_D\left(\mathbf{y}_D^t, \mathbf{y}_A^t, \mathbf{u}_D^t\right) = \underbrace{\|\mathbf{y}_D^t - \mathbf{y}_A^t\|_{Q_D}^2}_{\text{term1}} + \underbrace{\left\|\mathbf{u}_D^t\right\|_{R_D}^2}_{\text{term2}} \tag{2}$$

where $Q_D \in \mathbb{R}^{n\times n}$ and $R_D \in \mathbb{R}^{n\times n}$ are positive definite matrices. *Term1* prompts the defender to approach the attacker to intercept, while *term2* takes into account the cost of movement.

Attacker: The aim is to approach the protected area while avoiding being intercepted by the defender, so the cost function is defined as

$$J_A\left(\mathbf{y}_A^t, \mathbf{y}_D^t, \mathbf{u}_A^t\right) = \underbrace{\|\mathbf{y}_A^t - \mathbf{y}_p^t\|_{Q_A^1}^2}_{\text{term1}} - \underbrace{\left\|\mathbf{y}_A^t - \mathbf{y}_D^t\right\|_{Q_A^2}^2}_{\text{term2}} + \underbrace{\left\|\mathbf{u}_A^t\right\|_{R_A}^2}_{\text{term3}} \tag{3}$$

where $Q_A^1 = \omega_A^1 \mathbf{I}_n \in \mathbb{R}^{n\times n}$, $Q_A^2 = \omega_A^2 \mathbf{I}_n \in \mathbb{R}^{n\times n}$, and $R_A \in \mathbb{R}^{n\times n}$ are positive definite coefficient matrices. *Term1* encourages the attacker to approach the protected area's position, while *term2* motivates him to avoid the defender, and *term3* takes into account the cost of movement. We assume $\omega_A^1 > \omega_A^2$ in (3), which means that the attacker's intention to invade the protected area is stronger than the intention to avoid the defender.

III. Game Formulation

In this section, we will discuss the construction of an AD game model with level-k rationality.

A. Receding Horizon Optimization Problem

The players select optimal strategies in a receding horizon manner to enhance robustness with online feedback. Continuous replanning over a longer time horizon H while executing only over a shorter horizon allows the defender to react to the attacker's dynamic behavior.

For player $j \in V$, the receding horizon optimization problem with a prediction horizon of H steps is represented as follows:

$$\min_{\mathbf{u}_j^{\cdot|t}} \quad \sum_{i=1}^{H} J_j\left(\mathbf{y}_j^{t+i|t}, \mathbf{y}_{-j}^{t+i|t}, \mathbf{u}_j^{t+i|t}\right) \tag{4a}$$

$$\text{s.t.} \quad \mathbf{y}_j^{t+i} \in \mathcal{Y}_j, \mathbf{u}_j^{t+i} \in \mathcal{U}_j, \mathbf{x}_j^{t+i} \in \mathcal{X}_j \tag{4b}$$

$$\left(\mathbf{x}_j^{t+i}, \mathbf{u}_j^{t+i}\right) \in \mathcal{C}_j. \tag{4c}$$

where the decision variables $\mathbf{u}_j^{\cdot|t}$ represent the future H-step input sequence for player j at decision time t, given by $\mathbf{u}_j^{\cdot|t} = \{\mathbf{u}_j^{t+1|t}, \mathbf{u}_j^{t+2|t}, \ldots, \mathbf{u}_j^{t+H|t}\}$. Equation (4b) represents the input and state constraints, and (4c) represents the player dynamics constraint (1). Here, we consider $\mathbf{u}_j^{\cdot|t}$ as the decision variable, because given the state $\mathbf{x}_j^t$ and the input $\mathbf{u}_j^t$ at time t, the player's position at the future H-step can be derived. For brevity, we will use $\mathcal{J}_j\left(\mathbf{u}_j^{\cdot|t}\right)$ in the following text to represent the cost function (4a).

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

For the defender's optimization problem (4), when $j = D$, the cost function is coupled with the attacker's trajectory sequence $\mathbf{y}_A^{\cdot|t}$. To plan his own trajectory sequence $\mathbf{y}_D^{\cdot|t}$, the defender must first predict the attacker's trajectory sequence $\mathbf{y}_A^{\cdot|t}$. Therefore, we will use the game theory, explained in the following text, to predict $\mathbf{y}_A^{\cdot|t}$.

Assumption 1: The defender and the attacker know each other's current position and optimization problem.

B. Level-k Decision-Making Strategy

In this article, we adopt the level-k game theory to construct the interactive behavior of both the attacker and defender with bounded rationality.

In level-k game theory, the reasoning depth of player j is denoted by $k \in \mathcal{K}$, where $\mathcal{K}$ is a finite set defined as $\mathcal{K} = \{0, 1, \ldots, k_{\max}\}$. The reasoning level starts from level-0, where a player makes instinctive decisions to pursue its goal without considering the interactions with others. In contrast, a level-$k(k>0)$ player assumes that all other players are correspondingly at level-(k-1), taking into account interactions between players.

We assume that a level-0 player j believes that his opponent is stationary. This means that, after observing the state $\mathbf{y}_{-j}^t$ of his opponent at time t, the player assumes that $\mathbf{y}_{-j}^{t+i|t} = \mathbf{y}_{-j}^t$ for $i = 1, 2, \ldots, H$, and subsequently computes his decision $\mathbf{u}_j^t[0]$. In this setup, level-0 players have no reasoning ability and cannot predict the trajectories of their opponents.

We proceed to discuss the case where the level of reasoning is $k > 0$. This is a type of best response strategy where the player j will only adopt a strategy one level higher, i.e., level-k, if his opponent is a level-(k-1) player

$$\mathcal{J}_j(\mathbf{u}_j^{\cdot|t}[k]) = \sum_{i=1}^{H} J_j\left(\mathbf{y}_j^{i|t}[k], \hat{\mathbf{y}}_{-j}^{i|t}[k-1], \mathbf{u}_j^{i|t}[k]\right) \tag{5a}$$

$$\mathbf{u}_j^{\cdot|t}[k] = \arg\min_{\mathbf{u}_j \in \mathcal{U}_j} \mathcal{J}_j(\mathbf{u}_j^{\cdot|t}[k]) \tag{5b}$$

where $J_j\left(\mathbf{y}_j^{i|t}, \hat{\mathbf{y}}_{-j}^{i|t}[k-1], \mathbf{u}_j^{i|t}\right)$ represents the cost function of player j if the opponent player $-j$ adopts the level-(k-1) strategy $\hat{\mathbf{y}}_{-j}^{\cdot|t}[k-1]$, and $\mathbf{u}_j^{\cdot|t}[k]$ is the best response sequence for player j at level-k by solving the optimization problem (5b) subject to (4b) and (4c).

The obtained cost function for the strategy $\mathbf{u}_j^{\cdot|t}[k]$ at each step i over the horizon depends on the current observation states $\mathbf{y}_j^t$ and $\mathbf{y}_{-j}^t$, and the predicted trajectory sequence of the opponent $\hat{\mathbf{y}}_{-j}^{i|t}[k-1]$. To compute $\hat{\mathbf{y}}_{-j}^{\cdot|t}[k-1]$, player j constructs a prediction model for player $-j$ as

$$\mathcal{J}_{-j}(\hat{\mathbf{u}}_{-j}^{\cdot|t}[k-1]) = \sum_{i=1}^{H} J_{-j}\left(\hat{\mathbf{y}}_{-j}^{i|t}[k-1], \hat{\mathbf{y}}_j^{i|t}[k-2], \hat{\mathbf{u}}_{-j}^{i|t}[k-1]\right) \tag{6a}$$

$$\hat{\mathbf{u}}_{-j}^{\cdot|t}[k-1] = \arg\min_{\hat{\mathbf{u}}_{-j} \in \mathcal{U}_{-j}} \mathcal{J}_{-j}(\hat{\mathbf{u}}_{-j}^{\cdot|t}[k-1]). \tag{6b}$$

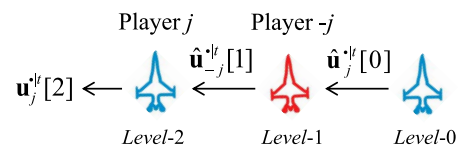


Fig. 2. Example of the player j's level-2 reasoning process. Player j assumes that player $-j$ is a level-1 decision maker, and player $-j$ will make decisions under the assumption that player j is a level-0 player. First, player j uses the observed state of player $-j$ to compute its own optimal trajectory at level-0, denoted as $\hat{\mathbf{u}}_j^{\cdot|t}[0]$. Next, player j predicts the optimal decision sequence of player $-j$ at level-1 with $\hat{\mathbf{u}}_j^{\cdot|t}[0]$, denoted as $\hat{\mathbf{u}}_{-j}^{\cdot|t}[1]$. Finally, player j computes his optimal trajectory at level-2 with $\hat{\mathbf{u}}_{-j}^{\cdot|t}[1]$, resulting in $\mathbf{u}_j^{\cdot|t}[2]$.

Player j's constructed model of strategy $\hat{\mathbf{u}}_j^{\cdot|t}[k-1]$ depends on the current observation states $\mathbf{y}_j^t$ and $\mathbf{y}_{-j}^t$, and its own predicted trajectory sequence $\hat{\mathbf{y}}_j^{i|t}[k-2]$. To compute $\hat{\mathbf{y}}_j^{i|t}[k-2]$, let $k = k-2$ in (5). The level-k reasoning process is hierarchical and recursive, and the depth of reasoning continues until the level is reduced to 0. So far, the level-k player j has predicted the level-$(k-1)$ strategy of player $-j$ and made his own decision.

We give an example to illustrate this level-k decision-making process. Suppose player j is at level-2, and Fig. 2 shows how the level-k decision-making mechanism works. This reasoning process allows player j to make decisions in a hierarchical and recursive manner, where each level builds upon the decisions and predictions made at the lower levels. Hence, if a player believes that his opponent (such as a bounded rational player) cannot deploy the NE but is using a level-k strategy, then the player has an incentive to respond with a level-$(k+1)$ strategy rather than a NE to exploit his opponent's weaknesses and maximize his own payoff. Under the interaction scheme at level-k (5), the best strategy is not necessarily a NE.

Remark 1: A NE strategy requires that all players are playing the same game and have the same decision-making mechanism, which can be a too strong setting in adversarial games. Furthermore, there may be multiple or infinite NEs in a game, and computing NE may be computationally intractable [8].

As described above, if we denote the set of player j best responses to an opponent strategy $\mathbf{u}_{-j}[k-1]$

$$\mathcal{BR}(\mathbf{u}_{-j}[k-1]) := \arg\min_{\mathbf{u}_j \in \mathcal{U}_j} \mathcal{J}_j(\mathbf{u}_j[k], \mathbf{u}_{-j}[k-1])\}$$

then the hierarchical best response structure in (5) can be equivalently represented as

$$\mathbf{u}_j[k] \in \mathcal{BR}\left(\mathbf{u}_{-j}[k-1]\right). \tag{7}$$

By constructing level-k policies, we have

$$\begin{aligned} \mathbf{u}_{-j}[k+1] &\in \mathcal{BR}\left(\mathbf{u}_j[k]\right) \\ \mathbf{u}_j[k+2] &\in \mathcal{BR}\left(\mathbf{u}_{-j}[k+1]\right). \end{aligned} \tag{8}$$

If $k \to +\infty$ and there exists $\mathbf{u}_j[k+2] = \mathbf{u}_j[k+1] = \mathbf{u}_j[k]$, then we have the following fixed point property:

$$\begin{aligned} \mathbf{u}_{-j}[k+1] &\in \mathcal{BR}\left(\mathbf{u}_j[k]\right) \\ \mathbf{u}_j[k] &\in \mathcal{BR}\left(\mathbf{u}_{-j}[k+1]\right). \end{aligned} \tag{9}$$

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

If the policies converge to the strategy pairs at a specific k level, the two players reach a NE by applying the joint policy $(\mathbf{u}_j[k], \mathbf{u}_{-j}[k+1])$.

Remark 2: Based on the discussion above, any convergent level-k strategy with infinite level of reasoning is a NE. Furthermore, in cases where there are multiple NEs, it can not determine which equilibrium the level-k strategy will converge to, as this may depend on the choice of the level-0 strategy.

Assumption 2: The attacker adopts a fixed level of decision-making reasoning in the AD game. The attacker and the defender do not know each other's k level.

IV. Adaptive Decision Making Strategy Based on Bayesian Inference Algorithm

If player j knows that the reasoning level of player $-j$ is k and does not exceed the maximum level $k_{\max}$, then player j adopts the counter strategy $\mathbf{u}_j^{\cdot|t}[k+1]$. However, in practical adversarial situations, the defender cannot always know the attacker's decision level. Therefore, the defender must infer the opponent's level in real time based on observation.

The inference methods described in [23], [24], [25], [26] first construct the opponent's decision models at different k levels. Then the difference between the current state of the attacker at time t, denoted as $\mathbf{y}_A^t$, and the first predicted trajectory point $\hat{\mathbf{y}}_A^{t+1|t-1}[k]$ from each level-k model associated with the trajectories predicted at time $(t\text{-}1)$ is calculated. Finally, k^* is found by $k^* \in \arg\min_{k=\{1,2,\ldots\}} \|\mathbf{y}_A^t - \hat{\mathbf{y}}_A^{t+1|t-1}[k]\|$, and the belief in k^* increases as other values of k decrease.

This method works well for autonomous vehicles in highway [23], [24], [25], [26], because their modes of motion are restricted to the traffic rules, trajectories at different k levels are easy to characterize. For the game in the article, the players can move freely. It is difficult to identify the motion behavior at the k level with a one-step match between the prediction model and the actual behavior.

To better capture an attacker's behavioral features and infer the attacker's decision-making level, we propose an adaptive decision algorithm based on the Bayesian inference algorithm and the multimodel strategy. The specific steps are given as follows.

Step 1: The defender uses (5) and (6) to construct the attacker's level-k prediction models $\mathcal{J}_A(\mathbf{u}_A^{\cdot|t}[k])$ for $k = 0, 1, \ldots, k_{\max}$.

Step 2: At time t, the defender observes the attacker's current position $\mathbf{y}_A^t$ and predicts the attacker's future trajectory sequence $\hat{\mathbf{y}}_A^{\cdot|t}[k]$ at each k level by solving $\mathcal{J}_A(\mathbf{u}_A^{\cdot|t}[k])$.

Step 3: The defender collects historical prediction trajectory sequences and samples no more than $M(2 \le M \le H)$ of trajectory points from each k level decision model, as shown in Fig. 3. If the current time is $t \le M$, sample $(t-1)$ points from the sequence of $(t-1)$ previously predicted trajectories. If the current time is $t > M$, sample the trajectories of M points counted forward from the current time t. These sampled points are the predicted trajectory points of different past moments for the same future moments. The defender's estimate of the

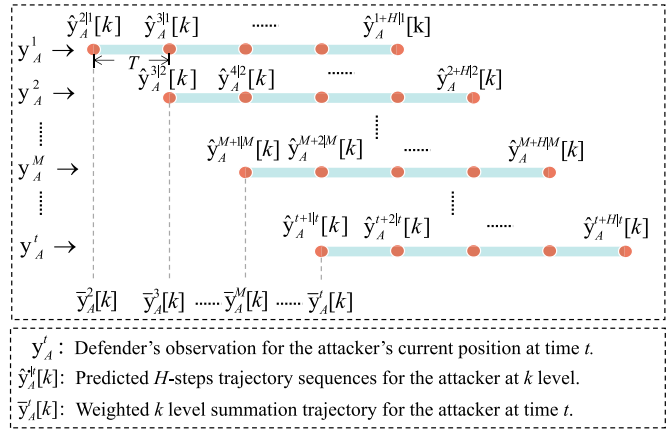


Fig. 3. Sample points in the historical prediction trajectory sequences. At time t, the defender observes the attacker's current position $\mathbf{y}_A^t$ and predicts the future trajectory sequence $\hat{\mathbf{y}}_A^{\cdot|t}[k]$ by solving the corresponding prediction model $\mathcal{J}_A(\mathbf{u}_A^{\cdot|t}[k])$ in (5) and (6). When $t \ge 2$, the defender calculates $\bar{\mathbf{y}}_A^t[k]$ based on the sample points from the historical prediction trajectory sequences.

level-k attacker's position at time t is obtained by summing the weighted trajectory points, which correspond to the predicted trajectory points at the same time in different prediction sequences (e.g., $\bar{\mathbf{y}}_A^3[k] = (1/3)\hat{\mathbf{y}}_A^{3|1}[k] + (2/3)\hat{\mathbf{y}}_A^{3|2}[k]$). If the trajectory point is sampled from a more recent prediction sequence, it has a higher weight than any sampled point from an earlier prediction sequence (e.g., in $\bar{\mathbf{y}}_A^3[k], \hat{\mathbf{y}}_A^{3|2}[k]$ has a higher weight than $\hat{\mathbf{y}}_A^{3|1}[k]$). The estimated mean position is expressed as follows:

$$\bar{\mathbf{y}}_A^t[k] = \begin{cases} \sum_{i=1}^{t-1} \frac{2^{i-1}}{2^{t-1}-1} \hat{\mathbf{y}}_A^{t|i}[k], 2 \le t \le M \\ \sum_{i=1}^{M} \frac{2^{i-1}}{2^{M}-1} \hat{\mathbf{y}}_A^{t|t-M+i-1}[k], M < t. \end{cases} \tag{10}$$

The corresponding standard deviation is then expressed as

$$\sigma_A^t[k] = \begin{cases} \sqrt{\frac{\sum_{i=1}^{t-1}\left(\hat{\mathbf{y}}_A^{t|i}[k] - \bar{\mathbf{y}}_A^t[k]\right)^2}{t-1}}, 2 \le t \le M \\ \sqrt{\frac{\sum_{i=t-M}^{t-1}\left(\hat{\mathbf{y}}_A^{t|i}[k] - \bar{\mathbf{y}}_A^t[k]\right)^2}{M}}, M < t. \end{cases} \tag{11}$$

Step 4: It is assumed that the defender uses the normal distribution $\mathcal{N}\left(\bar{\mathbf{y}}_A^t[k], (\sigma_A^t[k])^2\right)$ to describe the distribution of predicted trajectories under different k levels. We denote $\phi_k^t(\mathbf{y}_A^t | \bar{\mathbf{y}}_A^t[k], \sigma_A^t[k])$ as the likelihood function of the actual trajectory at level-k. We then update the belief that the attacker is at level-k based on the following Bayesian process:

$$\theta^{t+1}(k) = \frac{\theta^t(k)\phi_k^t(\mathbf{y}_A^t | \bar{\mathbf{y}}_A^t[k], \sigma_A^t[k])}{\sum_{k\in\mathcal{K}} \theta^t(k)\, \phi_k^t(\mathbf{y}_A^t | \bar{\mathbf{y}}_A^t[k], \sigma_A^t[k])} \tag{12}$$

where $\theta^t(k)$ represents the defender's belief that the attacker's level of reasoning is k at time t, and there satisfies $\sum_{k\in\mathcal{K}} \theta^t(k) = 1$.

Step 5: The defender's expected cost function is defined as the weighted sum of his own cost functions $\mathcal{J}_D(\mathbf{u}_D^{\cdot|t}[k+1])$

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Adaptive decision-making strategy with Bayesian inference for the defender.

Input: The maximum decision level $k_{\max}$; The initial beliefs $\theta^t(k)$ for $k = 0, 1, ..., k_{\max}$; The number of sample points M; The current observation $\mathbf{y}_A^t$ of the attacker.

Output: $\theta^{t+1}(k)$; $\mathbf{u}_D^{\cdot|t}$.

1: Construct the level-k prediction models $\mathcal{J}_A(\mathbf{u}_A^{\cdot|t}[k])$ for $k = 0, 1, ..., k_{\max}$ with (5) and (6);
2: Predict the attacker's future trajectory sequence $\hat{\mathbf{y}}_A^{\cdot|t}[k]$ with the current observation $\mathbf{y}_A^t$ by solving $\mathcal{J}_A(\mathbf{u}_A^{\cdot|t}[k])$ for $k = 0, 1, ..., k_{\max}$;
3: Compute (10) and (11) to estimate the mean value $\bar{\mathbf{y}}_A^t[k]$ and the standard deviation $\sigma_A^t[k]$;
4: Construct the normal distribution $\mathcal{N}\left(\bar{\mathbf{y}}_A^t[k], (\sigma_A^t[k])^2\right)$ for $k = 0, 1, ..., k_{\max}$ and compute the likelihood function $\phi_k^t(\mathbf{y}_A^t|\bar{\mathbf{y}}_A^t[k], \sigma_A^t[k])$, then update the belief $\theta^{t+1}(k)$ with (12);
5: Compute (13a) to obtain the expected cost function $\mathcal{J}_{\theta^{t+1}}(\mathbf{u}_D^{\cdot|t})$ with $\theta^{t+1}(k)$ and $\mathcal{J}_D(\mathbf{u}_D^{\cdot|t}[k+1])$, then compute (13b) to get the multi-model strategy $\mathbf{u}_D^{\cdot|t}$;
6: **return** $\theta^{t+1}(k)$, $\mathbf{u}_D^{\cdot|t}$.

for $k = 0, 1, ..., k_{\max}$. Finally, the defender obtains his adaptive multimodel strategy [26] based on the expected cost function:

$$\mathcal{J}_\theta(\mathbf{u}_D^{\cdot|t}) = \sum_{k\in\mathcal{K}} \theta^{t+1}(k)\mathcal{J}_D(\mathbf{u}_D^{\cdot|t}[k+1]) \tag{13a}$$

$$\mathbf{u}_D^{\cdot|t} = (1-a^t)\mathbf{u}_D^{\cdot|t-1} + a^t \arg\min_{\mathbf{u}_D\in\mathcal{U}_D} \mathcal{J}_\theta(\mathbf{u}_D^{\cdot|t}) \tag{13b}$$

where $a^t \in (0, 1]$ represents the step size of the strategy update, and the defender applies the first action $\mathbf{u}_D^{t+1|t}$ in a sequence $\mathbf{u}_D^{\cdot|t}$ over one time step.

In this article, we design the defence strategy from the defender's point of view. Thus, the defender uses Algorithm 1 to generate the interception trajectory at each moment, while the attacker uses a fixed k level strategy to generate the intrusion trajectory.

Remark 3: The proposed method works because it makes full use of historical information rather than single-step information. This leads to the evident differences between posterior and prior beliefs, which are so-called Bayesian surprise [28]. And the surprise is believed to be a necessary trigger for Bayesian learning. Continuous interactions with surprises in the AD game allow the posterior probabilities to be updated efficiently.

To summarize, the defender holds an initial belief about the attacker's model of reasoning level (levels-$0, 1, \ldots, k_{\max}$) as a prior probability distribution over all models. Then, based on the actual action applied by the attacker, the posterior probability distribution is updated. Finally, the defender computes his decisions based on both the updated probability distribution and the prediction associated with each model.

TABLE II
100 MONTE CARLO SIMULATION RESULTS AT DIFFERENT LEVEL-k

Interaction Type (Defender versus Attacker)	Results (Win : Lose)
$L0$ versus $L0$	82 : 18
$L1$ versus $L0$	90 : 10
$L1$ versus $L1$	80 : 20
$L2$ versus $L0$	75 : 25
$L2$ versus $L1$	94 : 6
$L2$ versus $L2$	82 : 18
NE versus $L0$	27 : 73
NE versus $L1$	38 : 62
NE versus $L2$	56 : 44

TABLE III
100 MONTE CARLO SIMULATION RESULTS UNDER DIFFERENT LEVEL-k

Interaction Type (Defender versus Attacker)	Results (Win : Lose)
Bayesian level-k versus $L0$	91 : 9
Bayesian level-k versus $L1$	94 : 6
Bayesian level-k versus $L2$	93 : 7

V. SIMULATIONS AND EXPERIMENTS

We consider $k_{\max} = 2$ and $\sum_{k=0}^{k_{\max}} \theta^t(k) = 1$ in the article.

A. Numerical Simulation Results

We first consider a 3-D (n = 3) game scenario, and perform numerical simulations to validate the proposed algorithm. The parameter settings of the game model are as follows. In the game, the prediction horizon is $H = 20$, and the sampling interval is $T = 0.15$. For cost matrices, $Q_D = 10 * \mathbf{I}$, $R_D = 5 * \mathbf{I}$, $Q_A^0 = 100 * \mathbf{I}$, $Q_A^1 = 90 * \mathbf{I}$, $R_A = 5 * \mathbf{I}$, where $\mathbf{I} \in \mathbb{R}^{n\times n}$ is the identity matrix. The initial positions of the players are set as follows. The defender's initial region is $[-5, 5, 2]^T \leq \mathbf{y}_D^0 \leq [5, 15, 2]^T$, while the attacker's initial region is $[15, 5, 2]^T \leq \mathbf{y}_A^0 \leq [25, 15, 2]^T$. The protected area is defined as a sphere with a center at $[-2, 15, 2]^T$ and a radius of 3. The defender's interception distance from the attacker is within 0.8. The defender's initial confidence in the attacker's decision level is set to $\theta^0 = [0.2, 0.5, 0.3]^T$. In the Bayesian learning process, we set $M = 15$ and the strategy update step size $a^t = 1$.

1) Simulation Results of Level-k Interactions: We test the interactions between the defender and the attacker for different combinations of reasoning levels. 100 Monte Carlo simulations are run for each type of interaction, and the results are summarized in Table II.

From Table II, we can see that in the case of level-k versus level-(k-1), i.e., when the defender is modeled as level-k and the attacker is modeled as level-(k-1), the defender has a high probability of winning, approaching 90%. In this case, the defender adopts the counter strategy. If the defender makes wrong assumptions about the attacker, for the cases of level-k versus level-k, and level-k versus level-(k-2), the defender gets a lower probability of winning. We also consider the case where the defender has a NE strategy in the game, but the attacker is a level-k player. The NE is computed by the IBR algorithm in

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

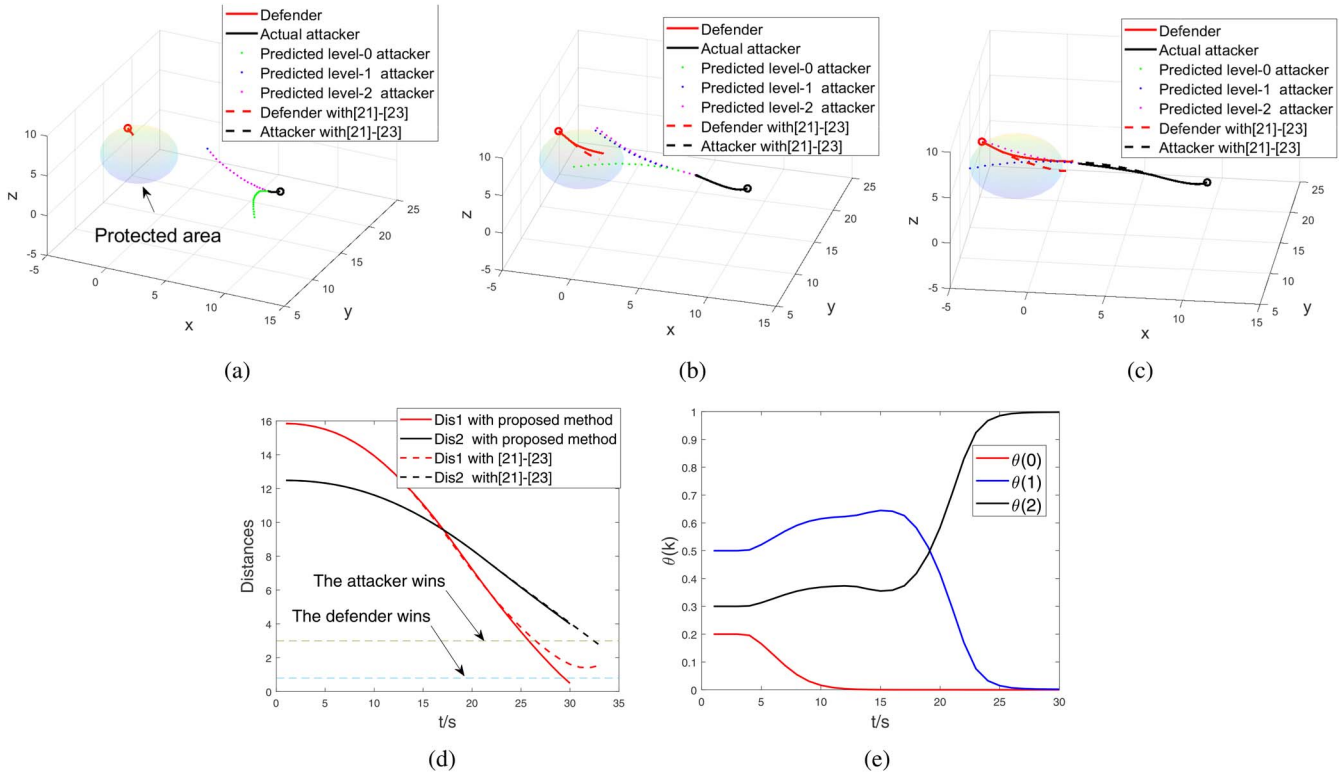


Fig. 4. Results of the simulation. (a)–(c) Game process of the defender and the attacker. (d) Distances between the defender and the attacker over time, the distances between the attacker and the protected area over time. (e) Process of belief update over time.

[8]. In the case of NE versus level-k, the defender also gets a lower probability of winning.

2) Simulation Results With Online Bayesian Learning: Consider an attacker with a fixed level of reasoning, and the defender does not know the attacker's reasoning level. The defender adopts the Bayesian inference algorithm and the multimodel strategy proposed in this article. The 100 Monte Carlo simulations with different level-k attackers are shown in the Table III. It can be seen that the defender has a similar win rate when facing attackers of different levels. Thus, the proposed method helps the defender to generate a counter-strategy when facing an opponent with an unknown level of reasoning.

One simulation result is shown in Fig. 4. In Fig. 4(a)–4(c), the red solid line represents the defender's trajectory, the black solid line represents the attacker's trajectory, the dashed lines represent the defender's predicted trajectories for different level-k prediction models of the attacker, and the dotted line represents the trajectory obtained using the inference algorithm from [23], [24], [25], [26]. In Fig. 4(d), we use "Dis1" to the represent the real-time distance between the defender and the attacker, we use "Dis2" to represent the real-time distance between the attacker and the protected area. The solid lines represent the distances obtained by the proposed method, and the dashed lines are the distances computed by the algorithm of [23], [24], [25], and [26]. The defender intercepts the attacker with our proposed technique at $t = 30$ s, while the attacker reaches the protected area with the algorithm of [23], [24], [25], and [26] at $t = 33$ s. The belief update process for different k levels is shown in Fig. 4(e). Within 1 to 10 s, the belief at the level-0 is directly reduced to 0. This can be observed from Fig. 4(a), where the predicted level-0 trajectory is completely different from the trajectory of the attacker. Within 1 to 20 s, the level-1 and level-2 beliefs increase together. From Fig. 4(a) and 4(b), the predicted level-1 trajectory coincides with the predicted level-2 trajectory. However, within 20 to 30 s, the level-2 belief converges to 1. In Fig. 4(c), only the predicted level-2 trajectory matches the attacker's motion trend. Therefore, the Bayesian inference algorithm captures the motion characteristics of the level-2 attacker, resulting in the convergence of the belief to the actual level.

3) Convergence Performance of the Bayesian Inference Algorithm: We demonstrate the convergence properties of the Bayesian inference algorithm proposed in this article. Different values of M are set to analyze the results of 50 Monte Carlo simulation experiments. We define that if the belief $\theta^{t_f}(k')$ for the level-k' attacker satisfies $\theta^{t_f}(k') \geq 0.5$ at the end time of the simulation t_f, then $\theta^{t_f}(k')$ has learned the correct level.

The statistical results are shown in Fig. 5, where the blue line represents the defender's winning percentage, and the red line represents the percentage of the correct convergence of $\theta^{t_f}(k')$. From the trend of the curve, we can see that the defender's win rate is positively correlated with the number of sample points M. When M is set to a sufficiently large value, the win rate increases rapidly and then stabilizes.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

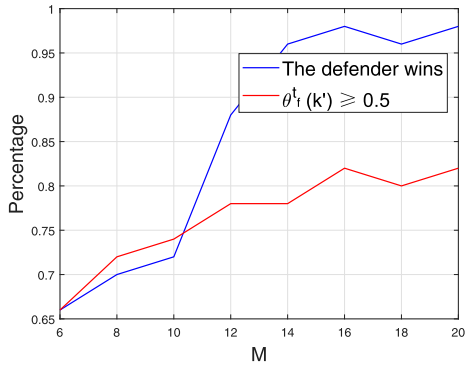


Fig. 5. Winning and convergence percentage with different sample number M.

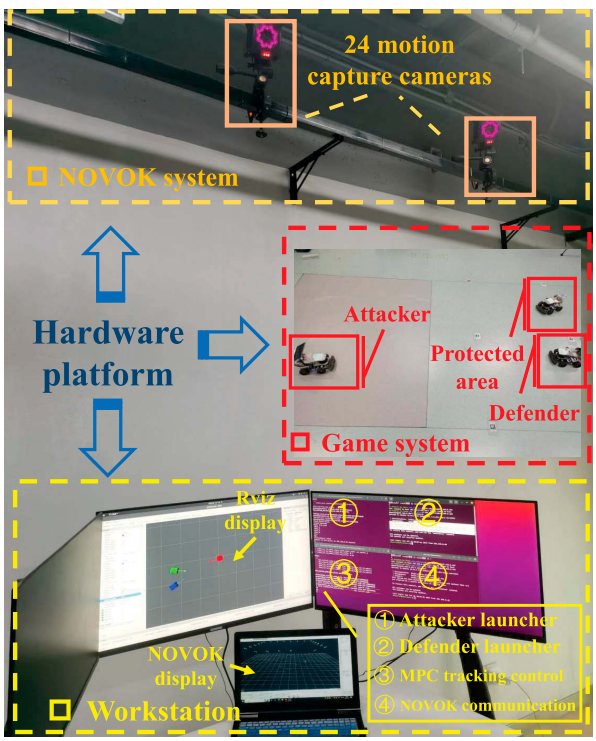


Fig. 6. Hardware platform.

Remark 4: There are many factors that affect the convergence performance of the Bayesian learning process, such as the duration of the game process, the initial setting of positions, and the estimated distribution of predicted trajectories at different k levels. It is difficult to analyze this performance theoretically, so we show the convergence performance through statistical results.

B. Experiment Results

To experimentally validate the effectiveness of the proposed algorithm, a hardware platform for the 2-D ($n = 2$) indoor AD game is set up as shown in Fig. 6.

The hardware platform was built with three main modules, the game system, the NOVOK system and the workstation. In the game system, three autonomous ground vehicles (AGVs) represent the attacker, the defender, and the protected area. In the NOKOV system, 24 motion capture cameras are used to capture the motion states of all the vehicles. The workstation preconstructs the motion capture map and runs the motion planning and control modules on an on-board Nvidia TX1 computer. The vehicles receive control signals via the ROS command *cmd_vel*.

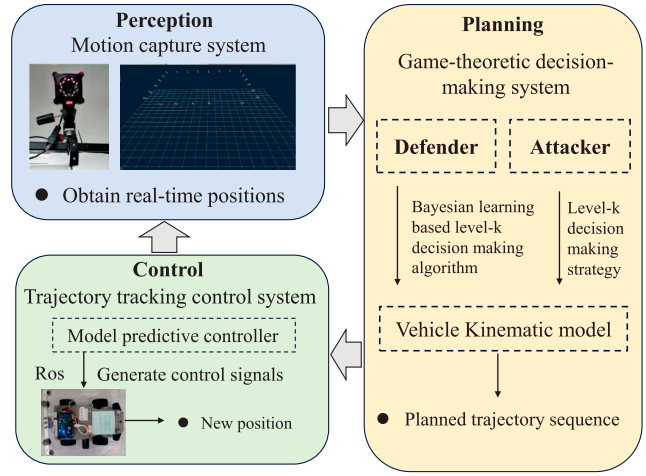


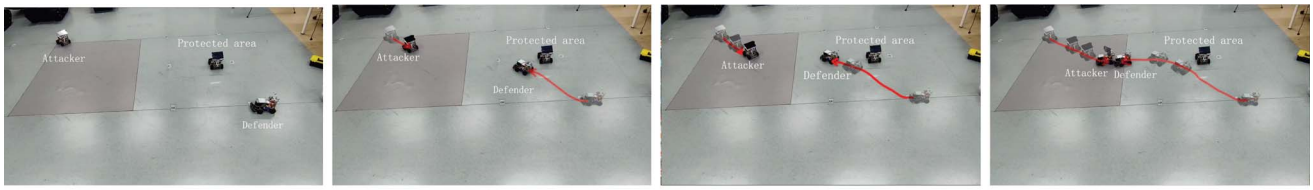
Fig. 7. Experiment framework.

The designed experiment framework is illustrated in Fig. 7. At each decision time, the following decision process takes place sequentially: In the perception module, the Nokov motion capture system obtains the positions of the attacker, the defender and the protected area, and transmits them to the attacker and the defender. In the planning part, the defender then uses the Bayesian learning algorithm to estimate the attacker's decision level and plans its interception trajectory accordingly. The attacker employs a fixed level-2 decision intelligence to plan its intrusion trajectory. Finally in the control module, a model predictive control (MPC) algorithm is used to track and control the receding horizon trajectory sequences generated by the attacker and defender. The generated control signals are executed by the built-in controllers of the AGversus Throughout the game, the above process runs in a loop until the game ends or the maximum simulation time is reached.

One of the experimental game process is depicted in Fig. 8, where the solid AGVs represent their current positions, while the transparent AGVs represent their positions in previous moments. The red lines indicate their motion trajectories. The experimental results show that the proposed algorithm enables the defender to estimate the attacker's reasoning level online and to plan effective interception trajectories.

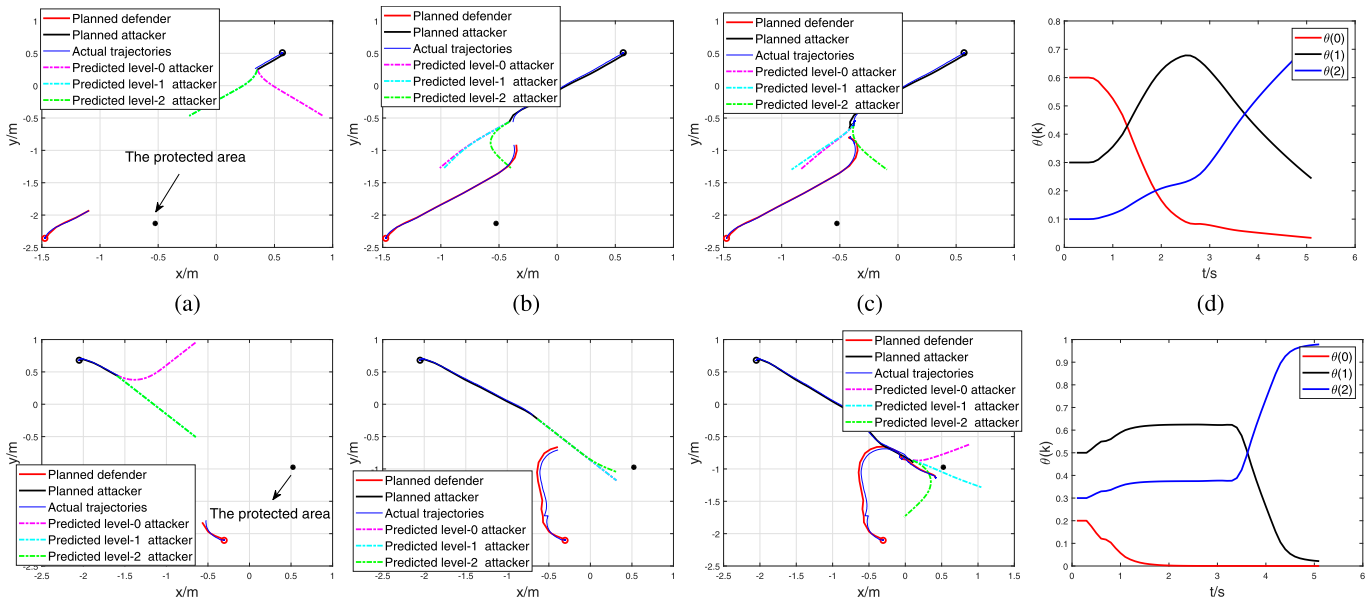
Figure 9 shows the results of the experiment. In Fig. 9(a)–9(c) and Fig. 9(e)–9(g), the red lines and black lines represent the planned trajectories of the defender and the attacker respectively, while the blue lines denote the actual trajectories. The dotted lines are the predicted level-k trajectories for the attacker. Fig. 9(d) and 9(h) show the belief learning process during the experiment. In this article, a linear dynamics model (1) is used

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 8. Experiment process based on Bayesian learning and the multimodel strategy. (a) $t = 0$ s (b) $t = 1.5$ s (c) $t = 3.6$ s (d) $t = 5.1$ s.



(a) (b) (c) (d) (e) (f) (g) (h)

Fig. 9. Experiment results. (a)–(c) Game process of the defender and the attacker. The solid lines are the planned and actual trajectories, while the dotted lines are the predicted trajectories for the attacker at different levels. (d) Process of belief update over time. (e)–(h) Experiment results of another game with different initial positions and different initial beliefs.

in the proposed path planning algorithm, while the experimental AGVs are equipped with Ackerman dynamics. Therefore, there must be tracking errors in the designed experiment framework. According to the experiment results, the blue line does not deviate much from the red and black lines, which means that our proposed algorithm can be used for practical applications.

Moreover, the belief update process is also consistent with the game process, e.g., in Fig. 9(a), at the begging, the defender's predicted trajectories for the level-1 and level-2 attackers coincide with the actual movement trend of the attacker, while the predicted level-0 trajectory deviates from the actual trajectory. As a result, the belief in level-1 and level-2 increases, while the belief in level-0 decreases. At $t = 4$ s, the predicted level-2 trajectory matches the actual movement trend of the attacker, leading to a continuous increase in the belief for level-2. Readers can watch a video demonstration of the experiment by following the link[1].

VI. Conclusion

Unlike the NE strategies that have been the focus of previous research on adversarial games, this article addressed the problem of AD games involving players with bounded rationality. Instead of NE strategies, the level-k defender adopted best response strategies assuming that his opponent is a level-(k-1) player. In addition, without knowing the reasoning level of the attacker, we proposed a Bayesian inference algorithm based on historical trajectory prediction sequences and current observations, which was able to characterize the attacker's motion behavior and update the belief in the attacker's reasoning level online. An adaptive multimodel strategy was generated with the beliefs. Ultimately, numerical simulations and experiments validated the effectiveness of the proposed algorithm.

This article only considered one defender versus one attacker level-k decision-making strategies, future research will focus on multiplayer game systems with limited rational players and their practical applications.

References

[1] B. Zhang, C. Dou, D. Yue, J. H. Park, and Z. Zhang, "Attack-defense evolutionary game strategy for uploading channel in consensus-based

[1]Video demonstration: https://www.bilibili.com/video/BV1v84y1D7Jc/?spm_id_from=333.999.0.0&vd_source=374e27cc09b6b5c47a99a2d28f71664c

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.

secondary control of islanded microgrid considering DoS attack," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 821–834, Feb. 2022.

[2] F. Li, "Network security evaluation and optimal active defense based on attack and defense game model," in *Proc. Int. Conf. Distrib. Comput. Elect. Circuits Electron.*, 2023, pp. 1–7.

[3] I. E. Weintraub, M. Pachter, and E. Garcia, "An introduction to pursuit-evasion differential games," in *Proc. Amer. Control Conf.*, 2020, pp. 1049–1066.

[4] Y. Guan, D. Maity, C. M. Kroninger, and P. Tsiotras, "Bounded-rational pursuit-evasion games," in *Proc. Amer. Control Conf.*, 2021, pp. 3216–3221.

[5] S. Nardi, F. Mazzitelli, and L. Pallottino, "A game theoretic robotic team coordination protocol for intruder herding," *IEEE Robot Autom. Lett.*, vol. 3, no. 4, pp. 4124–4131, Oct. 2018.

[6] A. Pierson and M. Schwager, "Controlling noncooperative herds with robotic herders," *IEEE Trans. Robot.*, vol. 34, no. 2, pp. 517–525, Apr. 2018.

[7] L. Liang, F. Deng, M. Lu, and J. Chen, "Analysis of role switch for cooperative target defense differential game," *IEEE Trans. Autom. Control*, vol. 66, no. 2, pp. 902–909, Feb. 2021.

[8] M. Wang, Z. Wang, J. Talbot, J. C. Gerdes, and M. Schwager, "Game-theoretic planning for self-driving cars in multivehicle competitive scenarios," *IEEE Trans. Robot.,* vol. 37, no. 4, pp. 1313–1325, Aug. 2021.

[9] R. Spica, E. Cristofalo, Z. Wang, E. Montijano, and M. Schwager, "A real-time game theoretic planner for autonomous two-player drone racing," *IEEE Trans. Robot.*, vol. 36, no. 5, pp. 1389–1403, Oct. 2020.

[10] T. Basar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, vol. 23. Philadelphia, PA, USA: SIAM, 1999.

[11] P. Yi and L. Pavel, "Asynchronous distributed algorithms for seeking generalized Nash equilibria under full and partial-decision information," *IEEE Trans Cybern*, vol. 50, no. 6, pp. 2514–2526, Jun. 2020.

[12] W. Liu, J. Lei, P. Yi, and Y. Hong, "No-regret learning for repeated non-cooperative games with lossy bandits," *Automatica*, vol. 160, 2024, Art. no. 111455, doi: 10.1016/j.automatica.2023.111455.

[13] V. P. Crawford and N. Iriberri, "Level-k auctions: Can a nonequilibrium model of strategic thinking explain the winner's curse and overbidding in private-value auctions?" *Econometrica*, vol. 75, pp. 1721–1770, Oct. 2007.

[14] T. H. Ho and X. Su, "A dynamic level-k model in centipede games," *Manage. Sci.*, vol. 59, pp. 452–469, Nov. 2012.

[15] D. Fudenberg and D. Levine, "Learning in games," *Eur. Econ. Rev.*, vol. 42, pp. 631–639, May 1998.

[16] M. Tambe, *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge, U.K.: Cambridge Univ. Press, 2011.

[17] S. J. Brams, and D. M. Kilgour, *Game Theory and National Security*. Oxford, U.K.: Blackwell, 1988.

[18] B. A. Swanson, Z. E. Fuchs, and J. Shroyer, "Singular solutions of the optimal pursuer response to fixed control strategies of a dual-evader defensive team," in *Proc. Amer. Control Conf.*, 2021, pp. 1991–1998.

[19] I. Erev and A. E. Roth, "Predicting how people play games: Reinforcement learning in experimental games with unique, mixed strategy equilibria," *Amer. Econ. Rev.,* vol. 88, no. 4, pp. 848–881, 1998.

[20] C. F. Camerer, T. H. Ho, and J. K. Chong, "A cognitive hierarchy model of games," *Quart. J. Econ.,* vol. 119, no. 3, pp. 861–898, 2004.

[21] N. Li, D. Oyler, M. Zhang, Y. Yildiz, A. Girard, and I. Kolmanovsky, "Hierarchical reasoning game theory based approach for evaluation and testing of autonomous vehicle control systems," in *Proc. 55th Conf. Decis. Control*, 2016, pp. 727–733.

[22] A. Kanellopoulos and K. G. Vamvoudakis, "Non-equilibrium dynamic games and cyber–physical security: A cognitive hierarchy approach," *Syst. Control Lett.,* vol. 125, pp. 59–66, Mar. 2019.

[23] M. Yuan, J. Shan, and H. Schofield, "Scalable game-theoretic decision-making for self-driving cars at unsignalized intersections," *IEEE Trans. Ind. Electron.*, vol. 71, no. 6, pp. 5920–5930, Jun. 2024, doi: 10.1109/TIE.2023.3290255.

[24] G. S. Sankar and K. Han, "Adaptive robust game-theoretic decision making strategy for autonomous vehicles in highway, " *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14484–14493, Dec. 2020.

[25] S. Karimi and A. Vahidi, "Receding horizon motion planning for automated lane change and merge using Monte Carlo tree search and level-k game theory, " in *Proc. Amer. Control Conf.*, 2020, pp. 1223–1228.

[26] N. Li, I. Kolmanovsky, A. Girard, and Y. Yildiz, "Game theoretic modeling of vehicle interactions at unsignalized intersections and application to autonomous vehicle control," in *Proc. Amer. Control Conf.*, 2018, pp. 3215–32202.

[27] A. G. Cunningham, E. Galceran, R. M. Eustice, and E. Olson, "MPDM: Multipolicy decision-making in dynamic, uncertain environments for autonomous driving," in *Proc. IEEE Int. Conf. Robot*, 2015, pp. 1670–1677.

[28] O. Catal, S. Leroux, C. De Boom, T. Verbelen, and B. Dhoedt, "Anomaly detection for autonomous guided vehicles using Bayesian surprise," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Las Vegas, NV, USA, 2020, pp. 8148–8153.

Hongwei Fang received the B.E. degree in automation from Wuhan Polytechnic University of Information Technology, Wuhan, China, in 2017, and the M.S. degree in control theory and control engineering from Sichuan University, Chengdu, China, in 2020. He is currently working toward the Ph.D. degree in control science and engineering with Tongji University, Shanghai, China.

His research interests include the multiplayer games and its applications.

Peng Yi received the B.E. degree in automation from the University of Science and Technology of China, Hefei, China, in 2011, and the Ph.D. degree in operations research and cybernetic from Academy of Mathematics and Systems Science, Beijing, China, in 2016.

He is a Professor with the Department of Control Science and Engineering, Tongji University, Shanghai, China. He was a Postdoctoral Fellow with the University of Toronto, Toronto, ON, Canada, from 2016 to 2017, and a Postdoctoral Associate with Washington University, St. Louis, MO, USA, from 2017 to 2019. His research interests include distributed optimization and game, neural systems and autonomous agents.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on January 12,2026 at 07:24:46 UTC from IEEE Xplore. Restrictions apply.